

Design Note — LLD Practice Platform

What the MVP is, how a learner moves through it, the classes that carry the work, and the trade-offs I made and would defend.

1. The MVP

One loop, done properly: pick an LLD problem → write a structured design → get criterion-by-criterion feedback that quotes your own words → try again as a new attempt → see whether you moved.

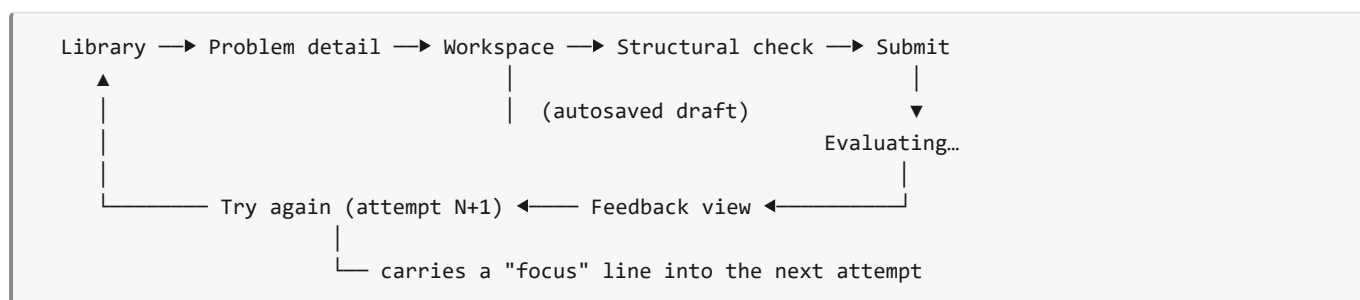
Scoped in:

- 4 problems (Parking Lot, Vending Machine, Elevator, Movie Ticket Booking)
- A 10-section structured submission format
- An 8-criterion weighted rubric, published to the learner before they write
- Two evaluators behind one interface: a deterministic offline one and an LLM one
- Attempt history with a progress comparison across attempts on the same problem

Scoped **out**, deliberately: accounts and auth, diagram editing, code submission, collaboration, payments, an admin panel. Each was cut because it adds surface without testing the core question, which is whether *explainable design feedback* is useful enough to come back for.

The riskiest assumption is not "can we score a design" — it is "will a learner trust the score enough to act on it". So the MVP spends its complexity budget on **attribution and transparency** (evidence quotes, published rubric, visible evaluator identity) rather than on breadth of content.

2. User flow



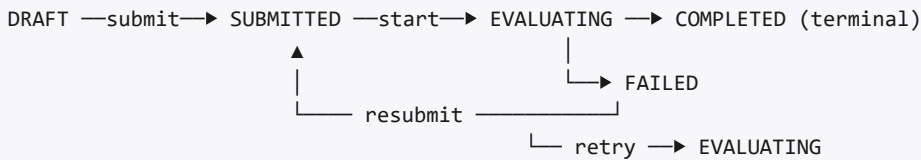
Four screens: `/library`, `/problems/:id/detail`, `/attempts/:id/workspace` and feedback, `/history`.

Two details that carry most of the product value:

The structural check is separate from judgement. Before anything is sent anywhere, deterministic rules ask: are the required sections present and substantial enough to be judged? It is instant, free, and returns exactly which checks failed. An unevaluable submission is rejected with `422` and a list — never with a vague low score. This means the expensive non-deterministic step is never asked to do a word count can do.

"Try again" creates attempt N+1 rather than editing attempt N. `COMPLETED` is a terminal state. History is immutable, which is the only thing that makes the progress comparison honest, and it is why `carriedFocus` — one line of "work on this next" — is threaded from one attempt's feedback into the next attempt's workspace.

The lifecycle, in one table



`FAILED` keeps the submission intact and offers a retry. A model timeout or a provider outage costs the learner nothing they wrote.

3. Classes

Layered so that dependencies point inwards. `domain/` imports nothing from Express, SQLite, or any model provider.

Domain

Class	Owns
<code>Attempt</code>	Aggregate root. The lifecycle of one try at one problem
<code>AttemptStatus</code>	The state machine: legal transitions in one table
<code>SubmissionContent</code>	(<i>interface</i>) What was submitted, in its native shape
<code>StructuredTextSubmission</code>	The one implemented format: 10 keyed sections
<code>DesignDocument</code>	The format-neutral view every evaluator reads
<code>SubmissionContentFactory</code>	Payload/stored JSON → content, one registry per format
<code>StructuralRules</code>	Deterministic evaluability checks
<code>Rubric / RubricRegistry</code>	The 8 weighted criteria and what good looks like
<code>Score</code>	A 0–10 value object with banding (<code>weak</code> ... <code>strong</code>) and <code>gap</code>
<code>CriterionFeedback</code>	One criterion's score, band, evidence and how to improve
<code>Evaluation</code>	Aggregates criteria into an overall score using rubric weights
<code>Evaluator</code>	(<i>port</i>) <code>supports(format)</code> + <code>evaluate(request)</code>
<code>Problem</code>	A problem definition and its requirements

`Attempt` is the aggregate root because every state change must be guarded in one place. No service, route or repository can put an attempt into an impossible state — evaluating a draft, editing a submitted design, completing twice — because the only way to change status is a method on `Attempt` that consults `canTransition`.

What `Attempt` deliberately does **not** own: how a design is judged (evaluators), what "good" means (`Rubric`), or how it is stored (repositories).

`Score` is a value object rather than a bare `number` because the scale, the rounding and the band language are domain decisions that the UI, both evaluators and the progress calculation all have to agree on. Putting them in one class is what stops "7.5" meaning three different things in three places.

Application

Class	Role
<code>PracticeService</code>	Use cases: start attempt, save draft, check structure, submit
<code>EvaluationCoordinator</code>	Runs evaluations in the background, with a timeout
<code>EvaluatorRegistry</code>	Dispatches to the first evaluator supporting the format
<code>AttemptHistoryService</code>	History and the across-attempt progress comparison

These coordinate; they do not decide. Nothing in `application/` names a model, an HTTP client or a provider.

Infrastructure and interfaces

`SqliteRepositories` + `mappers` (persistence), `config` (all env reading, once), `problems` (content), `interfaces/http` (Express routes + DTOs), and `composition-root.ts` — the single file that chooses which concrete implementation satisfies each port.

4. The two change tests

I judged the structure by asking what two plausible next features would actually cost. Both are named in the source comments so the seams are inspectable rather than claimed.

Change Test A — add a new submission format (e.g. a class diagram)

The seam is `DesignDocument`. Evaluators, structural rules, rubric scoring and the feedback UI all read `DesignDocument`, never a format. A new format implements `SubmissionContent.toDesignDocument()` (flattening nodes and edges into sections) and registers a `SubmissionCodec` in `SubmissionContentFactory` — one registry serving both directions, payload-in and stored-JSON-out.

Cost: one new class plus one codec registration. Nothing downstream changes. `SubmissionFormat` already names `class-diagram` and `code` so the seam's real cost is explicit rather than hypothetical.

Change Test B — add a new evaluator (rule-based, second model, or a human reviewer)

The seam is the `Evaluator` port. `PracticeService` and `EvaluationCoordinator` know only that interface. Implementations must return feedback for every criterion in the supplied rubric and throw rather than invent one when they cannot.

Cost: one new class plus one line in `resolveEvaluators` in the composition root. Nothing in `domain/` or `application/` changes, because nothing there names an evaluator.

Note what `EvaluatorVerdict` deliberately omits: no overall score, no ranking. Those are derived by `Evaluation` from the rubric, so every evaluator — human or machine — is aggregated identically and cannot grade itself generously.

5. Trade-offs

Structured text, not a diagram editor. A diagram tool is the "real" artefact of LLD and I chose against it. Structured sections give enough structure to attribute feedback to a place, without asking someone to learn a modelling notation during a 35-minute exercise. *Cost:* relationships are described in prose, so the evaluator reasons about stated dependencies rather than verified ones. `Change Test A` is the planned exit.

Two evaluators instead of one. Carrying an offline evaluator alongside the LLM one is real extra code. It buys three things, in order: anyone can clone the repo and see the whole loop work with no API key; tests exercise a *real* evaluator rather than a stub that proves nothing; and it forces the deterministic/judgement split to stay honest — everything the heuristic evaluator can do is, by construction, something the LLM should not be asked to do. It is labelled as pattern-based in the UI and never reports `high` confidence, because a demo that quietly pretends to be AI is worse than one that says what it is.

`node:sqlite` over `better-sqlite3` or `Prisma`. No native build step, no codegen, so `npm install && npm run dev` works on a clean machine with no toolchain. *Cost:* a Node

= 22.5 floor, and a smaller API than an ORM. For a single-writer local app that is the right side of the trade.

Background evaluation, not a blocking request. `POST /submission` returns `202` and the client polls. No HTTP connection is held open for a model call and `LLM_TIMEOUT_MS` bounds the work. *Cost:* the client needs polling and the UI needs real pending states — which it needs anyway to be honest about latency.

No accounts. A learner id is generated into `localStorage` and sent as an `x-learner-id` header. Enough to make attempt history real without building auth the product has not yet earned. *Cost:* history is per-browser and trivially spoofable. It is the first thing to replace if this goes past a prototype.

Idempotent submit. A double-clicked button or a retried request carrying the *same* content returns `already-submitted` rather than starting a second evaluation; different content mid-evaluation is a genuine conflict and is rejected. This is a correctness decision that costs a little branching in `Attempt.submit`, and it is the difference between a demo that survives an impatient user and one that bills twice.

Rubric weights live in the domain, not in a prompt. Class responsibilities carries the highest weight (1.4) because it is what LLD interviews actually turn on. Keeping weights out of the prompt means both evaluators are aggregated against the same definition, and changing the rubric does not mean re-tuning a string.

6. Limitations

- Prose relationships are not verified against the declared classes.
- One rubric version; no per-problem rubric weighting.
- The heuristic evaluator rewards structure and specificity, so a well-organised but wrong design scores better than it should. This is why it is labelled and capped.
- No auth, so history is per-browser.
- Single-node in-process evaluation queue; it does not survive a restart.